Software Enginnering – Cours 3 : Architecture

Architecture : Définir à un haut niveau les interactions entres les différentes composants du systèmes.

Principes :

- Loi de Conway
  « Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure. »

  La structure organisationnelle de l'entreprise va influencer l'architecture de leur système. On peut rarement designer une une architecture sans prendre en compte les interactions au sein de l'organisation.

  Peut poser des problèmes d'innovation car on reste coincé dans un schéma.

- S.O.L.I.D
  - Single-responsability principle: un composant doit avoir 1 objectif, 1 responsabilité

  - Open-closed principle : « Software entities … should be open for extension, but closed for modification. »
    Les objets ne doivent pas permettre de changer facilement leur implémentation interne mais doivent pouvoir étendre leur comportement.

  - Liskov substituion principle : « Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.»
    (plus bas niveau) Changer un objet par un autre si et seulement s'ils respectent le même contrat. C'est-à-dire que l'on peut changer un composant par un autre seulement s'ils gardent la même sémantique.
    Ex : *Sauvegarder les transactions d'une carte de crédit vers une base de donnée ou bien quelque part d'autre → pas de changement du point de vue externe → pas de changements pour le consommateur.*

  - Dependeny inversion principle : On dépend de l'abstraction et non pas de la façon dont c'est implémenté.

- YAGNI
  'You aren't gonna need it'
  On va toujours over enginnering (problème de biais et d'expérience) parce qu'on veut designer le système parfait.
  Ne pas créer un élément qui n'est pas nécessaire en pensant que l'on en a besoin plus tard.
  Quand est-ce qu'on s'arrête ?
  Il faut trouver l'équilibre entre développer ce qui est nécessaire pour pouvoir l'étendre ensuite, et le rendre assez simple

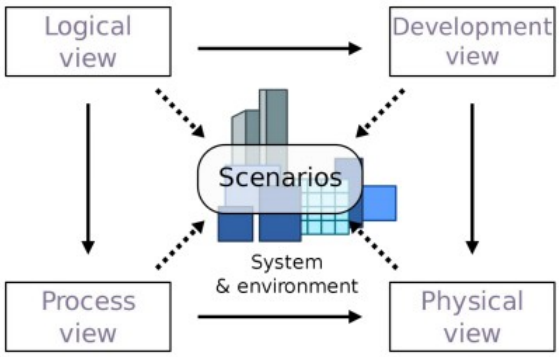
- KISS
  'Keep it simple, stupid'
  « The KISS principle states that most systems work best if they are kept simple rather than made complicated. Therefore, simplicity should be a key goal in design, and unnecessary complexity should be avoided. »

  Se poser la question : "Pourquoi je fais ça ?" afin d'éviter de faire certains design features pour de mauvaises raison.
  State of mind du programmeur : être fainéant ! Le moins d'effort → le moins de maintenance ensuite (en général).

- Loi de Déméter
  (lien avec le principe SOLID) Un composant doit seulement savoir ce qu'il doit connaître pour lui-même.
  Ex : *Encapsulation*
  Il est important de suivre ce type de principe mais si on en abuse, ça amène à des créations inutile de wrappers.

Concept de Quasi-décomposabilité : en regardant un système on peut analyser ses parties indépendamment les unes des autres.

Vues :



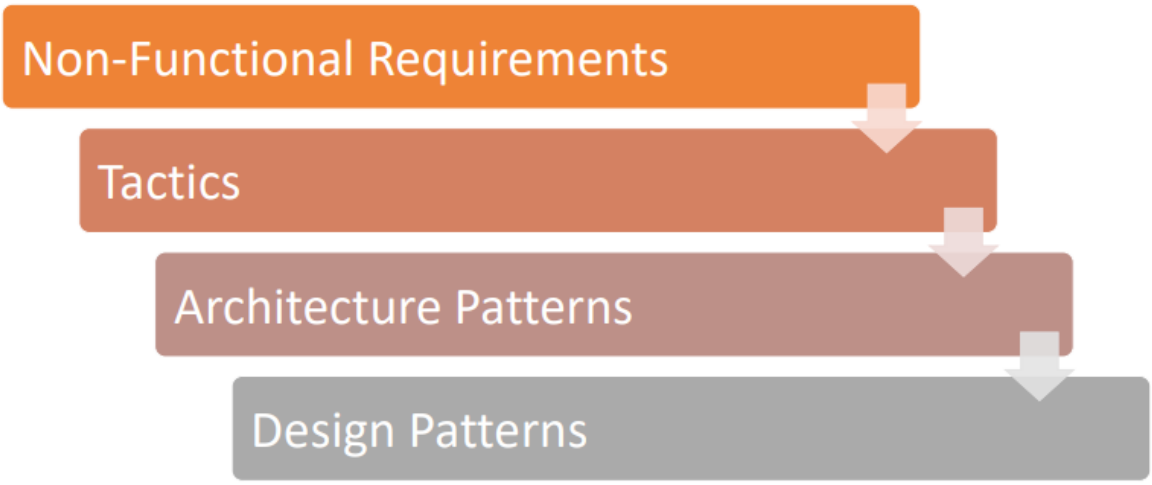
- Logical view : statique. Modèle du système. Quelles sont les données du systèmes ? → Diagramme de classe, diagramme d'état
- Process view : dynamique. Comment le système évolue ? Comment le système passe d'un état à un autre ? → Diagramme de séquence, diagramme d'activité
- Development view : Regarder le système en tant que comment ça se passe pour moi en tant que développeur ? Comment le packager ? Comment le développer jour après jour ?
- Physical view : Comment faire tourner le système ? Sur quelles infrastructures ?

- Scénario : Répondre au besoin du client. Donne un haut level de compréhension du but du système (use-case).

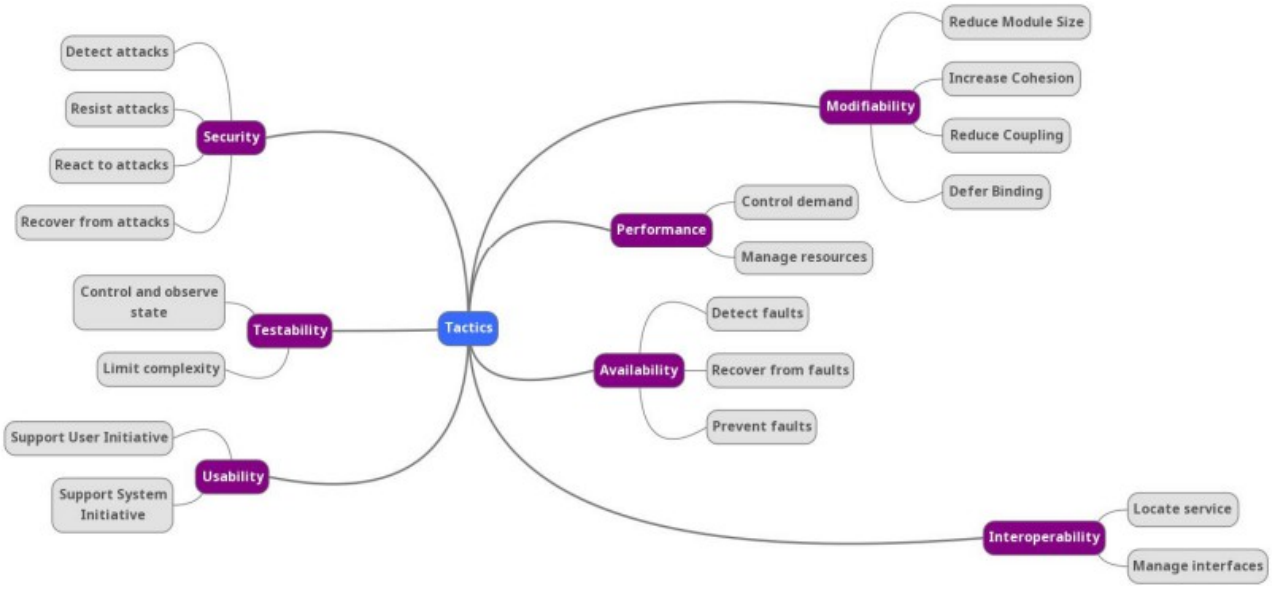
Les quatre premiers vues impactent les schémas livrés au client (n°5).
Ex : *Le développeur prend un certains nombre de décisions dans la Physical View qui fait qu'il n'est pas possible de déployer le système sur le Cloud → Est-ce que c'est ok pour le client ? Comment ça l'affecte ?*

Il est nécessaire dans les *Views* de ne pas trop aller dans les détails sinon la vue sera fausse. Quand on va trop vite dans les détails, on va trop vite s'attacher à une solution. Il est important de rester à un haut niveau d'abstraction dans la première phase d'architecture.
Il aussi nécessaire de choisir un format de communication et d'y rester mais de veiller à être compris par ses paires.
Ex : *Utiliser UML et utiliser des ad hoc à côté pour expliquer mais pas utiliser UML sans respecter son formalisme.*

Software Architecture Process



Tactics

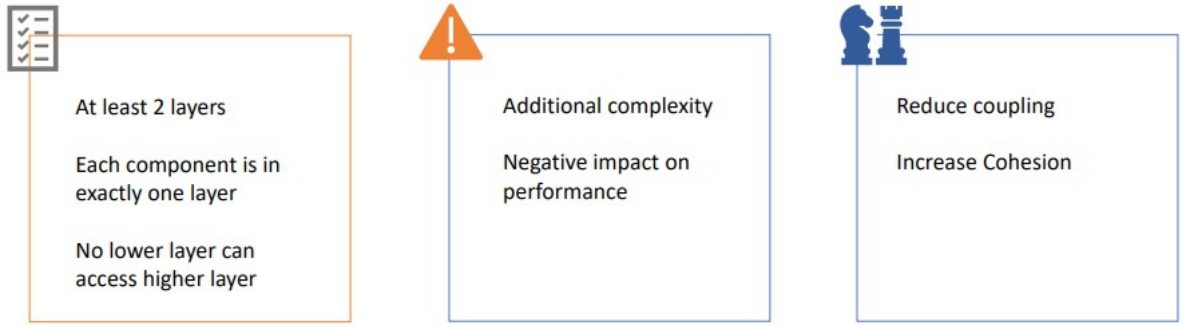


Note : toutes les tactics ne vont pas les unes avec les autres.

- Modifiability
  - Reduce module size : Plus les modules sont petits, plus ils sont simples à comprendre, plus ils sont faciles à modifier.
  - Increase cohesion : On ne mélange pas des choses qui n'ont rien avoir entre elles.
    Ex : *Un élément qui enregistre les transactions avec un autre qui calcule les balances.*
  - Reduce Coupling : Il est nécessaire d'isoler les composants des uns des autres : ils ne doivent pas avoir des connaissances internes des autres composants.
  - Defer Binding : Ne pas lier les composants entre eux au moment du développement, c'est au runtime qu'on décide.
- Performance
  Soit on réduit la demande soit on manage les ressources.
  - Control demand
  - Manage ressources
- Availability
  Faults = crash test etc
  - Detect faults
  - Recover from faults
  - Prevent faults
- Interoperability
  Le systèmes est capable de communiquer / d'opérer avec d'autres systèmes (pas forcément avec les mêmes technologies).
  - Locate service
  - Manges interfaces
- Security
  - Detect attacks (le plus important, parce qu'un système a de grandes probabilité de se faire attaquer)
  - Resist attacks
  - React to attacks
  - Recover from attacks
- Testability
  Si on peut contrôler l'état dans lequel est notre système et savoir dans lequel il est, on peut alors tester facilement.
  - Control and observe state
  - Limit complexity
- Usability
  - Support User Initiavtive : permettre à l'utilisateur de faire ce qu'il a envie de faire. Ex : *Interface graphique super facile d'utilisation.*
  - Support System Initiative : permettre au système de faire de nouvelles choses. Ex : *API facile à utiliser.*

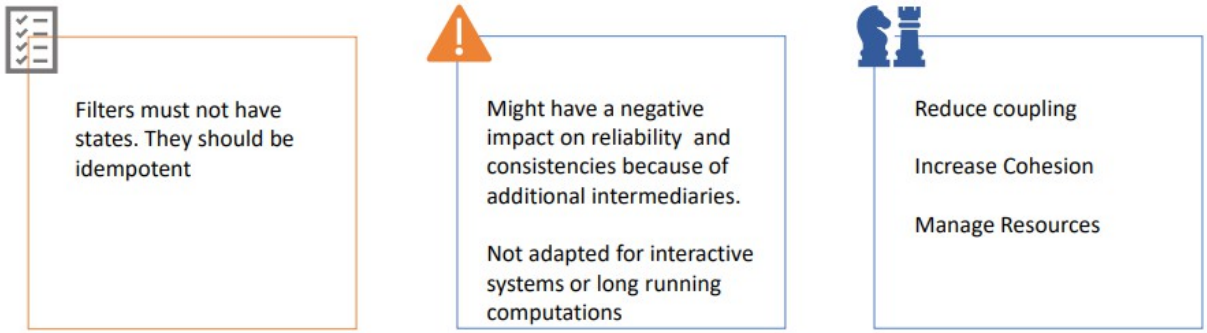
Pattern d'Architecture :

- Layered
  Systèmes en couches, chaque composant est liée à une seule couche. (cf quasi-décomposabilité)
  Une couche ne peut communiquer qu'avec la couche d'en dessous sinon ça créerait un cycle et il n'y aurait plus la décomposabilité.
  Ex : *Modèle OSI (open system interconnection)*
  - Réduit le couplage car il y a des contrats entre les différentes couches
  - Augmente la cohésion car une couche ne peut communiquer qu'avec une seule couche.

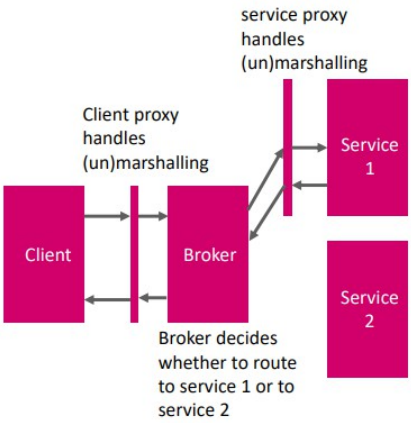


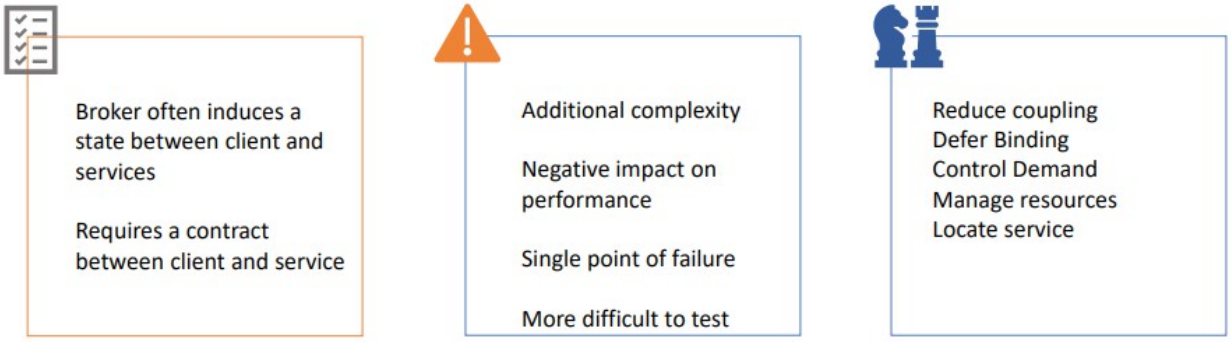
- Pipe and Filters
  Défini une séquence de transformation en un flux d'informations.
  Les filtres doivent être idempotent → toujours avoir la même sortie → pas d'état dans le filtre (ex : *pas d'accumulateur*).
  - ++ pour paralléliser des processus
  - -- 1 infos à la fois
  - -- stabilité du système : plus on a de pipes, plus il y a des risques que ça casse
  - ++ réduit le couplage car la communication entre chaque filtre est clairement défini

  Ex : *Compilateurs*

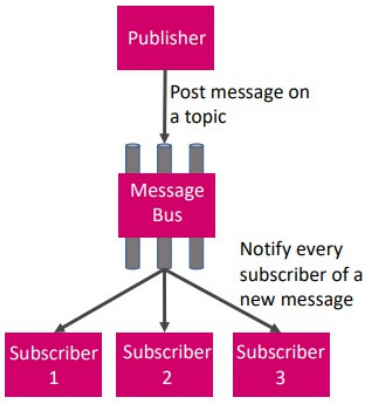


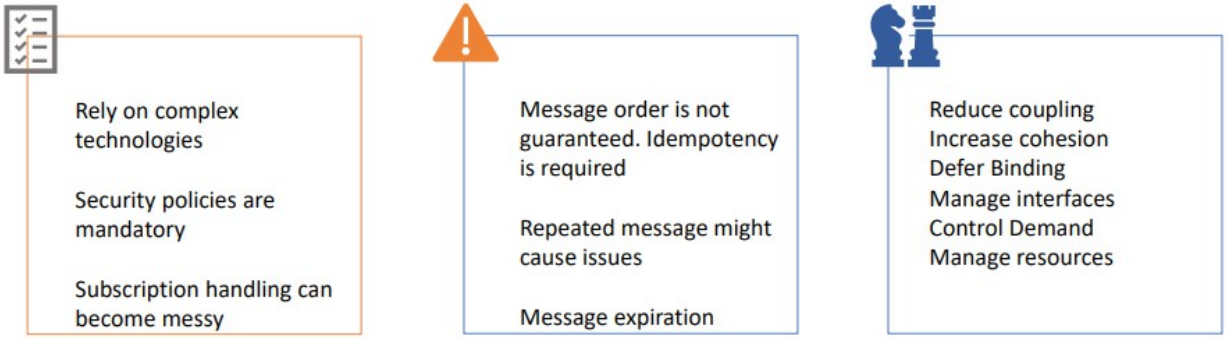
- Broker
  Intermédiaire entre un client et un service. On sépare le consommateur du fournisseur.
  Ex : *Composant hardware : un routeur*
  Besoin d'avoir un état au milieu, idéalement il devrait être stocké chez le client et soit passé ensuite à chaque requête. Une autre solution est d'avoir la synchronization des états entre les deux. Le broker pourrait aussi toujours envoyer le client vers la même source mais s'il est défectueux → problème et cela implique que le broker doit garder en mémoire où l'envoyer.
  Il est essentiel d'avoir un contrat entre le client et le service.





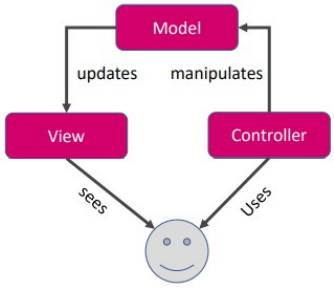
- Pusbilsh-Subscribe
  Architecture de type asynchrone. Quelqu'un publie un message et l'utilisateur le consomme quand il le souhaite. Celui qui envoie et celui qui reçoit ne se connaissent pas.
  Il est nécessaire d'avoir le même contrat entre les deux : être d'accord sur la forme du messsage.

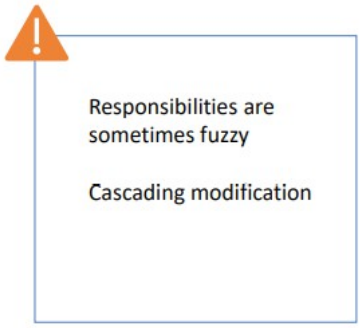


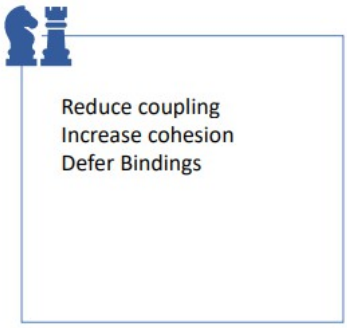


- Model View Controller
  Défini pour les interfaces graphiques. Cet architecture contient 3 modèles :
  - Un modèle qui contient les données à afficher.
  - Une vue qui contient la présentation de l'interface graphique.

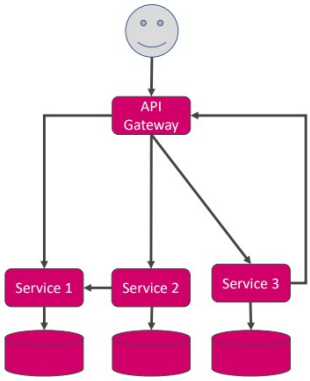
- Un contrôleur qui contient la logique concernant les actions effectuées par l'utilisateur.

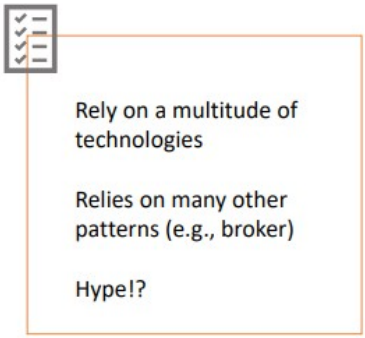


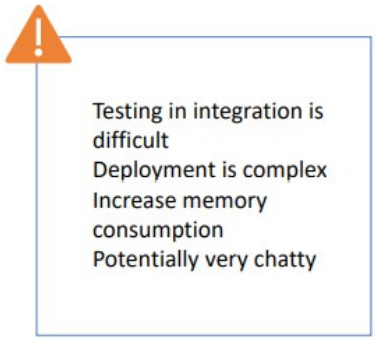


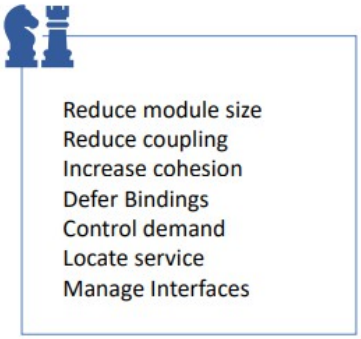


- Micro-services
  Développer une grand nombre de petits services qui fonctionnent ensembles en synergie pour accomplire les mêmes tâches.

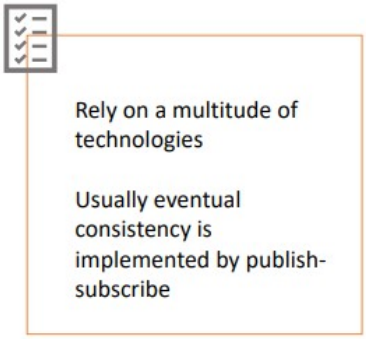


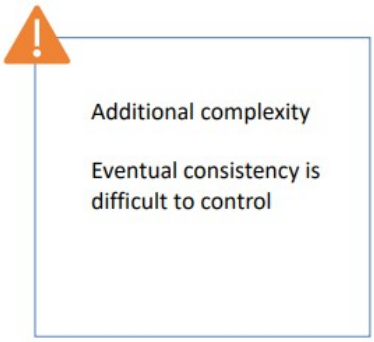


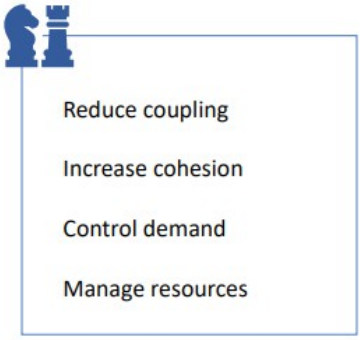


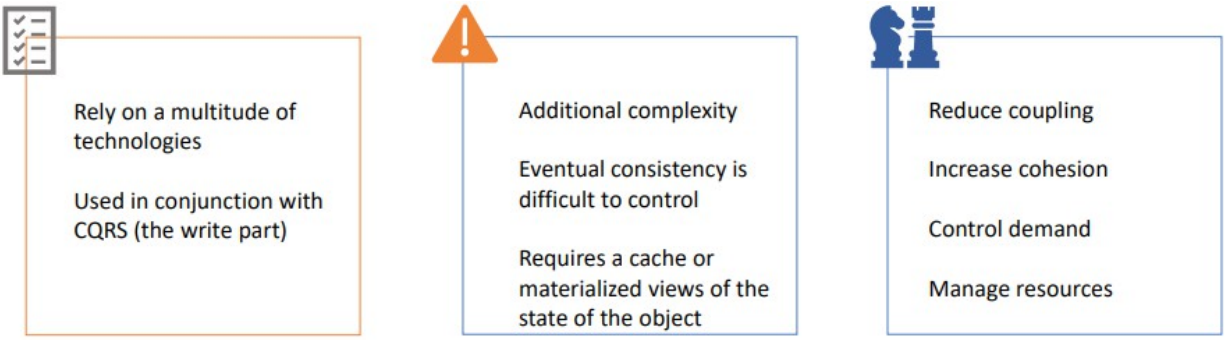


- Command Query Responsability Segregation
  Architecture qui permet de séparer la lecture de l'écriture







- Event Sourcing

Ici il n'est pas question de sauvegarder un état mais de sauvegarder une séquence d'événement qui amène à un état.